

Lab Task 1: Stock Price Monitor using AVL Tree

You are building a **real-time stock price monitoring system** for a trading platform.

The platform continuously receives updates about stocks — new ones being added, existing ones changing price, and occasionally being removed.

The system must support:

- **Fast search and updates** by stock ID.
- **Automatic rebalancing** to ensure performance remains consistent even under thousands of updates.
- **Analytical queries** such as getting all stocks within a price range or identifying the highest and lowest priced stocks.

Since stock data arrives dynamically and unpredictably, the system uses an **AVL Tree** keyed by **stockID** to maintain **$O(\log n)$** efficiency for insertions, deletions, and lookups.

Data Model

Each node in the AVL tree represents one stock record.

```
struct Stock {
    int stockID;      // Unique key for stock (e.g., 101)
    char name[50];    // Stock name
    float price;      // Current stock price
};

struct Node {
    Stock data;
    Node* left;
    Node* right;
    int height;
};
```

Functional Requirements

1. **Insertion** — **addStock()**

- Inserts a new stock into the AVL tree using `stockID` as the key.
- Duplicate IDs are **not allowed** — reject duplicates and return `false`.
- Automatically rebalances the tree after insertion.

2. Update — `updatePrice()`

- Updates the `price` of an existing stock.
- Must preserve the AVL property.
- If stock not found, return `false`.

3. Deletion — `removeStock()`

- Removes a stock node by `stockID`.
- Properly rebalances the tree after deletion.

4. Find — `findStock()`

- Returns a pointer to the stock record matching the ID.
- Returns `nullptr` if not found

5. Range Query — `getStocksInPriceRange()`

- Returns all stocks whose `price` lies in `[minPrice, maxPrice]`.
- Output via dynamically allocated array (created with `new`), and sets `outCount` to the number of results.

6. Highest/Lowest Queries

- `getHighestPricedStock()` returns the stock with the maximum price.
- `getLowestPricedStock()` returns the stock with the minimum price.
- If the tree is empty, return a dummy Stock with `stockID = -1`.

7. Diagnostics

- `isBalanced()` : verifies AVL property (height difference ≤ 1 for all nodes).

- `getHeight()` : returns the tree's height.
- `getTotalStocks()` : returns the total number of stocks currently stored.

8. Display

- `displayInorder()` : prints stock IDs in ascending order for debugging.

Test Cases

```
#include "gtest/gtest.h"
#include "StockAVL.h"

// 1. Insert and Find Stock
TEST(StockAVLTest, AddAndFindStock) {
    StockAVL avl;
    EXPECT_TRUE(avl.addStock(101, "Tesla", 750.5));
    EXPECT_TRUE(avl.addStock(102, "Apple", 180.2));

    Stock* s = avl.findStock(101);
    ASSERT_TRUE(s != nullptr);
    EXPECT_STREQ(s->name, "Tesla");
    EXPECT_FLOAT_EQ(s->price, 750.5);
}

// 2. Reject Duplicate Stock IDs
TEST(StockAVLTest, RejectDuplicateStockIDs) {
    StockAVL avl;
    EXPECT_TRUE(avl.addStock(100, "Microsoft", 320.1));
    EXPECT_FALSE(avl.addStock(100, "Meta", 240.0));
    EXPECT_EQ(avl.getTotalStocks(), 1);
}

// 3. Update Existing Stock Price
TEST(StockAVLTest, UpdateStockPrice) {
    StockAVL avl;
    avl.addStock(1, "NVIDIA", 100.0);
    EXPECT_TRUE(avl.updatePrice(1, 120.0));

    Stock* s = avl.findStock(1);
```

```

    ASSERT_TRUE(s != nullptr);
    EXPECT_FLOAT_EQ(s->price, 120.0);
}

// 4. Update Non-Existing Stock Fails
TEST(StockAVLTest, UpdateNonExistingStock) {
    StockAVL avl;
    avl.addStock(5, "Intel", 35.0);
    EXPECT_FALSE(avl.updatePrice(10, 90.0)); // no stock with ID 10
}

// 5. Auto-Balancing After Multiple Inserts (LR Rotation)
TEST(StockAVLTest, AutoBalancingAfterInsert) {
    StockAVL avl;
    avl.addStock(30, "A", 10.0);
    avl.addStock(10, "B", 20.0);
    avl.addStock(20, "C", 30.0); // triggers LR rotation
    EXPECT_TRUE(avl.isBalanced());
}

// 6. Delete Stock and Maintain Balance
TEST(StockAVLTest, DeleteAndRebalance) {
    StockAVL avl;
    avl.addStock(50, "A", 10.0);
    avl.addStock(40, "B", 15.0);
    avl.addStock(60, "C", 20.0);

    EXPECT_TRUE(avl.removeStock(40));
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_EQ(avl.getTotalStocks(), 2);
}

// 7. Remove Non-Existing Stock
TEST(StockAVLTest, RemoveNonExistingStock) {
    StockAVL avl;
    avl.addStock(10, "A", 100.0);
    EXPECT_FALSE(avl.removeStock(99));
    EXPECT_EQ(avl.getTotalStocks(), 1);
}

```

```

// 8. Find Highest and Lowest Priced Stocks
TEST(StockAVLTest, HighestLowestStock) {
    StockAVL avl;
    avl.addStock(1, "X", 50.0);
    avl.addStock(2, "Y", 100.0);
    avl.addStock(3, "Z", 75.0);

    Stock high = avl.getHighestPricedStock();
    Stock low  = avl.getLowestPricedStock();

    EXPECT_STREQ(high.name, "Y");
    EXPECT_STREQ(low.name, "X");
}

// 9. Get Stocks in Price Range
TEST(StockAVLTest, StocksInPriceRange) {
    StockAVL avl;
    avl.addStock(1, "A", 10.0);
    avl.addStock(2, "B", 50.0);
    avl.addStock(3, "C", 70.0);
    avl.addStock(4, "D", 100.0);

    int count = 0;
    Stock* results = avl.getStocksInPriceRange(40.0, 80.0, count);
    EXPECT_EQ(count, 2); // Stocks B and C
    EXPECT_TRUE(results[0].price >= 40.0 && results[0].price <= 80.0);
    EXPECT_TRUE(results[1].price >= 40.0 && results[1].price <= 80.0);
    delete[] results;
}

// 10. Tree Height After Multiple Insertions
TEST(StockAVLTest, HeightAfterInsertions) {
    StockAVL avl;
    for (int i = 1; i <= 15; ++i)
        avl.addStock(i, "S", (float)(i * 10));
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 8);
}

// 11. Deletion Sequence Maintains Balance

```

```

TEST(StockAVLTest, SequentialDeletionsBalanced) {
    StockAVL avl;
    for (int i = 1; i <= 10; ++i)
        avl.addStock(i, "P", (float)(i * 5));
    for (int i = 2; i <= 10; i += 2)
        avl.removeStock(i);
    EXPECT_TRUE(avl.isBalanced());
}

```

// 12. Delete All Stocks

```

TEST(StockAVLTest, DeleteAllStocks) {
    StockAVL avl;
    avl.addStock(10, "A", 20.0);
    avl.addStock(20, "B", 30.0);
    avl.addStock(30, "C", 40.0);

    avl.removeStock(10);
    avl.removeStock(20);
    avl.removeStock(30);

    EXPECT_EQ(avl.getTotalStocks(), 0);
    EXPECT_TRUE(avl.isBalanced());
}

```

// 13. Empty Tree Highest/Lowest

```

TEST(StockAVLTest, EmptyTreeEdgeCase) {
    StockAVL avl;
    Stock high = avl.getHighestPricedStock();
    Stock low = avl.getLowestPricedStock();
    EXPECT_EQ(high.stockID, -1);
    EXPECT_EQ(low.stockID, -1);
}

```

Lab Task 2: Dynamic Median Tracker Using AVL Tree

You are designing a **real-time median tracking system** for a financial analytics platform.

The system continuously receives **streaming numbers** — for example, stock prices, sensor readings, or incoming data packets.

After each insertion or deletion, the system must be able to **instantly report the current median** of all values seen so far.

A simple array or list would make median finding too slow ($O(n \log n)$ after sorting).

Instead, your task is to implement this using an **AVL Tree**, which maintains balance and enables efficient **rank-based lookups**.

The AVL must be *augmented* so that each node also stores the **size of its subtree** — allowing the median to be located in $O(\log n)$ time.

Data Model

Each node represents one numeric data point.

```
struct Node {
    float value;    // The data value (e.g., 55.3)
    int height;    // For AVL balancing
    int subtreeSize; // Count of all nodes in this subtree (including itself)
    Node* left;
    Node* right;
};
```

Note:

Every rotation and insertion must also update **subtreeSize**, so it always reflects the total number of descendants under a node.

That way, you can navigate by **rank** to find the median position.

Functional Requirements

1. `insertValue(float val)`

- Inserts a new value into the AVL tree.
- Balances the tree after insertion.

- Updates both `height` and `subtreeSize` on every return step.
- Returns `true` on success, `false` if value already exists.

2. `deleteValue(float val)`

- Removes a node with the given value.
- If not found, returns `false`.
- Rebalances and updates `subtreeSize` afterward.

3. `getMedian()`

- Returns the **current median value** in $O(\log n)$ time.
- For odd number of nodes: middle element.
- For even number of nodes: average of the two middle elements.
- Uses a **k-th smallest element** helper based on subtree sizes:

4. `getCount()`

- Returns total number of values (root's `subtreeSize`).

5. `isBalanced()`

- Verifies AVL property ($|height(left) - height(right)| \leq 1$ for all nodes).

6. `displayInorder()`

- Prints all elements in ascending order (for debugging).

Median Calculation

Let n = number of values in tree.

- If n is odd $\rightarrow$ median = value at position $(n + 1) / 2$
- If n is even $\rightarrow$ median = average of values at positions $n / 2$ and $(n / 2 + 1)$

```
#include "gtest/gtest.h"
#include "MedianAVL.h"

// 1. Simple Median (Odd Count)
TEST(MedianAVLTest, SimpleOddMedian) {
    MedianAVL avl;
    avl.insertValue(10);
    avl.insertValue(30);
    avl.insertValue(20);
    EXPECT_FLOAT_EQ(avl.getMedian(), 20.0f);
}

// 2. Median After Even Inserts
TEST(MedianAVLTest, SimpleEvenMedian) {
    MedianAVL avl;
    avl.insertValue(10);
    avl.insertValue(20);
    avl.insertValue(30);
    avl.insertValue(40);
    EXPECT_FLOAT_EQ(avl.getMedian(), 25.0f); // avg(20,30)
}

// 3. Auto-Balancing After Inserts
TEST(MedianAVLTest, BalancingAfterInsert) {
    MedianAVL avl;
    avl.insertValue(50);
    avl.insertValue(30);
    avl.insertValue(40); // triggers LR rotation
    EXPECT_TRUE(avl.isBalanced());
}

// 4. Delete Value Updates Median
TEST(MedianAVLTest, DeleteUpdatesMedian) {
    MedianAVL avl;
    avl.insertValue(10);
    avl.insertValue(20);
```

```

    avl.insertValue(30);
    avl.insertValue(40);
    avl.insertValue(50);
    EXPECT_FLOAT_EQ(avl.getMedian(), 30.0f);

    avl.deleteValue(30);
    EXPECT_FLOAT_EQ(avl.getMedian(), 35.0f); // avg(30,40) after deletion
}

// 5. Handle Duplicate Insertion Gracefully
TEST(MedianAVLTest, RejectDuplicates) {
    MedianAVL avl;
    EXPECT_TRUE(avl.insertValue(25.0f));
    EXPECT_FALSE(avl.insertValue(25.0f));
    EXPECT_EQ(avl.getCount(), 1);
}

// 6. Delete Nonexistent Value
TEST(MedianAVLTest, DeleteNonexistentValue) {
    MedianAVL avl;
    avl.insertValue(10);
    EXPECT_FALSE(avl.deleteValue(999)); // no node
    EXPECT_TRUE(avl.isBalanced());
}

// 7. Median After Many Inserts
TEST(MedianAVLTest, MedianAfterBulkInserts) {
    MedianAVL avl;
    float vals[] = {10, 20, 30, 40, 50, 60, 70};
    for (int i = 0; i < 7; ++i)
        avl.insertValue(vals[i]);
    EXPECT_FLOAT_EQ(avl.getMedian(), 40.0f);
}

// 8. Median After Bulk Inserts and Deletions
TEST(MedianAVLTest, BulkInsertAndDelete) {
    MedianAVL avl;
    for (int i = 1; i <= 10; ++i)
        avl.insertValue((float)i);
    EXPECT_FLOAT_EQ(avl.getMedian(), 5.5f);
}

```

```
        for (int i = 1; i <= 5; ++i)
            avl.deleteValue((float)i);
        EXPECT_FLOAT_EQ(avl.getMedian(), 8.0f);
    }

// 9. Check Subtree Size Integrity
TEST(MedianAVLTest, SubtreeSizesValid) {
    MedianAVL avl;
    for (int i = 1; i <= 5; ++i)
        avl.insertValue((float)i * 10);
    EXPECT_EQ(avl.getCount(), 5);
    EXPECT_TRUE(avl.isBalanced());
}

// 10. Empty Tree Edge Case
TEST(MedianAVLTest, EmptyTreeMedian) {
    MedianAVL avl;
    EXPECT_EQ(avl.getCount(), 0);
    EXPECT_FLOAT_EQ(avl.getMedian(), 0.0f); // convention: 0 for empty
}
```